LARAVEL AUTHENTICATION WITH JWT

Mid-Level Full-Stack Project

Simple JWT Login System with Frontend & Backend

Project Type:	Full-Stack Web Application
Backend:	Laravel + Sanctum JWT
Frontend:	React or Vue.js
Database:	MySQL
Duration:	40-50 hours

PROJECT OVERVIEW

Build a complete authentication system where users can register, login, and access a protected dashboard. The system uses JWT tokens for security and has role-based access (Student, Teacher, Admin).

What You Will Learn:
• User registration and login system
• JWT token generation and validation
• Protected API routes
• User profile management
• Role-based access control
• Frontend state management
• API integration with frontend

BACKEND TASKS (LARAVEL)

Task 1: Setup & Database

What to do:
1. Install Laravel Sanctum: `composer require laravel/sanctum`
2. Run migration: `php artisan migrate`
3. Create Users table with fields: name, email, password, phone, role
4. Create Roles table: id, name (Student, Teacher, Admin)
5. Create role_user pivot table for user-role relationship
6. Configure Sanctum in config/sanctum.php

Database Tables:
• users: id, name, email, password, phone, role, created_at
• roles: id, name
• role_user: user_id, role_id

Files to Create:
• Migration files for tables
• User model with relationships
• Role model

Task 2: User Registration & Login API

Create 4 API Endpoints:
1. POST /api/register
Accept: name, email, password, phone, role
Return: user data + access_token
Validate: email unique, password min 8 characters
2. POST /api/login
Accept: email, password
Return: user data + access_token
Validate: email exists, password matches
3. POST /api/logout
Accept: nothing (just requires token)
Revoke the token
Return: success message
4. GET /api/user (Protected Route)
Return: current user's profile data
Requires: Valid JWT token

Create These Controllers:
• AuthController - handle registration, login, logout
• UserController - handle user profile

Create Middleware:
• Middleware to verify JWT token on protected routes

Task 3: User Profile Endpoints

Create Profile Management Endpoints:
1. GET /api/user/profile
Return: Full user details (name, email, phone, profile_picture)
2. PUT /api/user/profile
Accept: name, email, phone
Update user information
Return: updated user data
3. POST /api/user/change-password
Accept: current_password, new_password

Validate: current password correct, new password min 8 chars
Return: success message

4. POST /api/user/upload-picture

Accept: image file (max 2MB)
Store in storage/uploads/profiles
Return: image URL
All require authentication (valid JWT token)

Task 4: Role-Based Access Control

Create Role-Based Middleware:

Create middleware to check user roles before accessing endpoints.

Admin Routes:

• GET /api/admin/users - List all users
• POST /api/admin/users/{id}/assign-role - Assign role to user
• DELETE /api/admin/users/{id} - Delete user

Teacher Routes:

• GET /api/teacher/dashboard - Teacher dashboard
• GET /api/teacher/courses - List courses

Student Routes:

• GET /api/student/dashboard - Student dashboard
• GET /api/student/courses - List enrolled courses

Protect Routes Like This:

```
Route::middleware('auth:sanctum', 'role:admin')
->get('/admin/users', ...)
```

Task 5: Test All Endpoints

Using Postman:

1. Test Registration
• POST /api/register with all fields
• Verify token is returned
• Test validation (duplicate email, weak password)
2. Test Login
• POST /api/login with valid credentials
• POST /api/login with wrong password (should fail)
• Verify token is returned
3. Test Protected Routes
• GET /api/user with valid token (should succeed)
• GET /api/user without token (should fail 401)
• GET /api/admin/users as admin (should succeed)
• GET /api/admin/users as student (should fail 403)
4. Test Profile Updates
• PUT /api/user/profile with new data
• POST /api/user/change-password with correct password
• POST /api/user/upload-picture with image file
5. Test Logout
• POST /api/logout to revoke token
• Try using same token (should fail)

FRONTEND TASKS (REACT/VUE.JS)

Task 1: Project Setup

Setup Steps:
1. Create React project: `npx create-react-app auth-app`
OR Vue project: `npm create vite@latest auth-app -- --template vue`
2. Install required packages:
• axios (for API calls)
• react-router-dom (React only, for routing)
3. Create folder structure:
```
src/
■■■ pages/ (Login, Register, Dashboard)
■■■ components/ (forms, buttons, navbar)
■■■ services/ (api.js for API calls)
■■■ context/ or store/ (AuthContext or Vuex)
■■■ App.js
```
4. Create .env file:
```
REACT_APP_API_URL=http://localhost:8000/api
```

Task 2: API Service Layer

Create services/api.js:
This file handles all communication with backend:
1. Create Axios instance with base URL
2. Add interceptor to include JWT token in all requests
3. Create functions for:
• register(name, email, password)
• login(email, password)
• logout()
• getProfile()
• updateProfile(data)
• changePassword(currentPassword, newPassword)
• uploadPicture(file)
4. Handle errors and 401 responses (redirect to login)
Token Storage:
Store token in localStorage when user logs in
Send token in every API request: `Authorization: Bearer {token}`
Clear token from localStorage on logout

Task 3: Authentication Pages

Create LoginPage Component:
Form with:
• Email input field
• Password input field
• Submit button
• "Forgot password?" link
• "Don't have account? Register" link
On submit:
1. Call api.login(email, password)
2. Save token to localStorage
3. Save user data to context/store
4. Redirect to /dashboard
5. Show error message if login fails

Create RegisterPage Component:
Form with:
• Name input
• Email input
• Password input
• Confirm password input
• Phone input (optional)
• Role dropdown (Student/Teacher)
• Submit button
• "Already have account? Login" link
On submit:
1. Validate all fields
2. Call api.register(...)
3. Save token and redirect to dashboard
4. Show error if registration fails

Task 4: Dashboard & Profile Pages

Create DashboardPage Component:
Display:
• Welcome message with user name
• User's profile picture
• Current role
• Quick action buttons:
- View Profile
- Change Password
- Logout
Create ProfilePage Component:
Display:
• User's full information (name, email, phone)
• Edit profile button
• Change password button
• Upload picture button
Allow:
• Click "Edit" to update name, email, phone
• Click "Change Password" to update password
• Click "Upload" to change profile picture
After update, refresh data from API

Task 5: Navigation & Protected Routes

Create Navigation Component:
Show:
• Logo/brand name
• Navigation links (Home, Profile, Admin if admin user)
• User name and picture
• Logout button
Create Protected Routes:
For React Router:
1. Create PrivateRoute component
2. Check if token exists
3. Check if user has required role
4. Allow or redirect to /login
Routes structure:
• /login - public
• /register - public

• /dashboard - protected (all roles)
• /profile - protected (all roles)
• /admin/users - protected (admin only)

Handle Token Expiration:

1. Check if token expired on each page load
2. If expired, redirect to login
3. Show message "Session expired, please login again"

Task 6: Error Handling & Loading States

Show Loading Spinner:

Display loading spinner while:
• API call in progress
• Page data loading
• Form submitting

Show Error Messages:

Display error notifications for:
• Network errors
• Validation errors from server
• 401 Unauthorized (redirect to login)
• 403 Forbidden (show "Access denied")
• 500 Server errors

Show Success Messages:

Display success toast after:
• Successful login
• Successful registration
• Profile update
• Password change
• Picture upload

Message auto-disappear after 3-5 seconds

INTEGRATION BETWEEN FRONTEND & BACKEND

API Response Format (Backend returns):
Success Response (200):
```
{
"success": true,
"user": { "id": 1, "name": "John", "email": "john@test.com" },
"access_token": "token_string"
}
```
Error Response (400+):
```
{
"success": false,
"message": "Invalid email or password"
}
```
Frontend sends Authorization Header:
`Authorization: Bearer eyJhbGc...`
Frontend handles Token Refresh:
If backend returns 401, call refresh token endpoint
If refresh succeeds, retry original request
If refresh fails, redirect to login

TESTING CHECKLIST

Backend Testing (Postman):
■ Register new user with valid data
■ Register fails with duplicate email
■ Register fails with weak password
■ Login with correct credentials
■ Login fails with wrong password
■ Token received on successful login
■ Logout revokes token
■ GET /api/user with token succeeds
■ GET /api/user without token returns 401
■ GET /api/admin/users as admin succeeds
■ GET /api/admin/users as student returns 403
■ Update profile works
■ Change password works
■ Upload picture works
Frontend Testing:
■ Registration page loads
■ Registration form validates (empty fields)
■ Registration submits and logs in
■ Login page loads
■ Login works with correct credentials
■ Login shows error with wrong password
■ Dashboard shows after login
■ User name appears on dashboard
■ Profile page loads and shows data
■ Edit profile updates data
■ Change password form works
■ Upload picture works
■ Logout clears token
■ Protected pages redirect to login when not authenticated
■ Admin pages only show for admin users

PROJECT DELIVERABLES

Item	Description
Backend Code	Laravel project with all API endpoints working
Database	MySQL with users, roles, and migrations
Frontend Code	React/Vue.js app with all pages and forms
API Testing	Postman collection with test requests
Documentation	README with setup and usage instructions
Git Repository	Clean commit history with meaningful messages

QUICK REFERENCE

Backend Stack: Laravel 10+ | MySQL | PHP 8.1+ | Sanctum JWT
Frontend Stack: React 18+ or Vue 3+ | Axios | React Router
Key Packages: laravel/sanctum, axios, react-router-dom
Main API Endpoints:
POST /api/register - Register new user
POST /api/login - User login
POST /api/logout - User logout
GET /api/user - Get user profile
PUT /api/user/profile - Update profile
POST /api/user/change-password - Change password
Database Tables:
users (id, name, email, password, phone, role)
roles (id, name)
role_user (user_id, role_id)
Frontend Pages:
/login - Login page
/register - Registration page
/dashboard - User dashboard
/profile - User profile page
/admin - Admin panel (admin only)